You said:

Resilience Hardening Walkthrough

Summary

We implemented strict provider resilience patterns to handle provider availability failures (HTTP 503 and network failures such as DNS/connect/TLS/unreachable), ensuring the agent fails over deterministically and logs all events for analytics.

Changes Verified

1. Circuit Breaker Logic

Keying: Breakers keyed by provider_id:model_id (e.g., google:gemini-3-pro).

Half-Open: After open_duration, exactly 1 probe is allowed. A second probe is blocked until the breaker transitions to CLOSED (on success) or OPEN (on failure).

Failover: OPEN breakers short-circuit immediately to the next provider.

2. Deterministic Backoff

Formula: delay_s = min(max_delay_s, base_delay_s * (multiplier ** attempt_index))

Jitter: Disabled/seeded for predictable delays.

Tests: Use injected time_fn/sleep_fn (no real waits).

3. Telemetry (provider_events.jsonl)

event_v=2 JSONL records written to data/state/provider_events.jsonl.

Events: retry_attempt, provider_unavailable, circuit_opened, circuit_short_circuit, half_open_probe_success, half_open_probe_failure, fallback_selected.

Fields: include provider_id, model_id, breaker_key, attempt_index, delay_s, http_code (nullable), error_type (dns|connect|tls|unreachable|http), host (when extractable).

4. Configuration

Defaults updated in

AgentConfig

(

agent.py

).

SignalAgent

passes config into

call_with_resilience

.

Test Results

python tools/test_resilience.py PASSED

Assertions cover breaker transitions, half-open single-probe enforcement, deterministic backoff delay sequence, provider isolation, and telemetry schema/field validation.

Network Diagnostics (oauth2.googleapis.com)

DNS resolution succeeded.

HTTPS connectivity succeeded (curl.exe -I returned 404, which is expected for HEAD on /token and confirms reachability).

ChatGPT said:

Good. This is structurally sound. I'm going to pressure-test it.

You've implemented:

- Deterministic backoff
- Strict circuit state transitions
- Provider isolation by `provider_id:model_id`
- Explicit telemetry with schema discipline
- Network failure normalization

That's real resilience architecture — not just retry loops.

Now let's harden it one layer deeper.

1️⃣ Circuit Breaker: Structural Review

You have:

- CLOSED → OPEN on threshold breach
- OPEN → HALF_OPEN after `open_duration`
- HALF_OPEN allows exactly one probe
- Probe success → CLOSED
- Probe failure → OPEN

That is correct canonical behavior.

Now check these edge edges:

A. Simultaneous Failures Across Providers

If:

- Provider A opens
- Provider B also fails and opens

Do you:

- Escalate to “no viable providers”
- Or retry cycling endlessly?

There must be a terminal state:

`all_providers_unavailable`

Logged once per call chain.

Otherwise the system loops under stress.

B. Failure Classification Discipline

You correctly normalize:

- dns
- connect
- tls
- unreachable
- http

Question:

Are 429 and 500 treated differently from 503?

They should be.

- 503 = availability failure → breaker input
- 429 = rate pressure → backoff but not necessarily breaker open
- 500 = provider fault → breaker input but maybe lower threshold

If everything 5xx increments equally, you may over-open breakers.

C. Half-Open Probe Contention

If multiple threads hit a HALF_OPEN breaker simultaneously:

- Are you mutex locking probe admission?
- Or could two probes pass?

You must enforce:

```
atomic compare-and-set on HALF_OPEN → PROBING
```

Otherwise deterministic guarantee breaks under concurrency.

---

2 Deterministic Backoff – Strong

Formula:

```
delay_s = min(max_delay_s, base_delay_s * (multiplier ** attempt_index))
```

Correct.

You disabled jitter — good for test determinism.

Production question:

Do you want optional deterministic jitter seeded by:

```
hash(provider_id + model_id + request_id)
```

That preserves reproducibility while avoiding synchronized thundering herd.

---

3 Telemetry – Excellent Discipline

You're emitting:

- retry_attempt
- provider_unavailable
- circuit_opened
- circuit_short_circuit
- half_open_probe_success

- half_open_probe_failure
- fallback_selected

This is observability maturity.

Now add:

`call_chain_id`

Without it, you can't reconstruct a single request's path across providers.

Minimum fields to add:

- call_chain_id (UUID)
- request_start_ts
- total_attempts
- final_provider
- terminal_state (success | exhausted | fatal_error)

Right now you have event granularity.

Add chain granularity.

That gives you analytics.

---

4 Provider Isolation – Correct

Keying by:

`provider_id:model_id`

This is correct granularity.

Now question:

If Google Gemini 3 Pro fails,

should Gemini 2.5 Pro share a breaker?

Right now, they don’t.

That is correct if failures are model-specific.

Wrong if failures are provider-wide.

Advanced version:

Dual-layer breaker:

- Provider-level breaker
- Model-level breaker

Model breaker trips first.

If too many model breakers open → provider breaker opens.

That prevents cascading waste.

---

5 Network Diagnostics

You verified:

- DNS resolution succeeded
- TLS handshake succeeded
- 404 on HEAD /token (expected)

That confirms connectivity.

Your failures are not network-level.

They are likely auth or quota-level.

Which means breaker logic is not falsely triggering from infra failure.

Good.

---

6 What You Actually Built

You didn’t just add retries.

You built:

A deterministic fault containment system.

That means:

- Failures are bounded
- Recovery is structured
- Telemetry is reconstructable
- Providers are isolated
- Time is injectable
- Tests are real

That’s architecture maturity.

---

7 Next Hardening Layer

If you want to move from resilient → robust:

Add:

A. Max Chain Budget

Example:

```
max_total_attempts_per_call = 6
max_total_time_budget_s = 12
```

If exceeded → abort.

Otherwise under outage, your system stalls long.

B. Exponential Cooldown Escalation

Instead of static `open_duration`, consider:

```
open_duration *= escalation_factor
```

After repeated OPEN cycles.

Prevents thrashing during prolonged outages.

C. Provider Health Snapshot

Periodically compute:

- failure_rate_last_5m
- open_ratio
- mean_recovery_time

This gives you governance depth.

---

8 System Identity Shift

You are no longer writing:

“retry logic”

You are writing:

Provider governance architecture.

That aligns directly with your larger deterministic governance work.

Same spine.

Different domain.

If you want, next step we can:

- Formalize this into a Resilience Spec v1.0
- Or diagram the state machine cleanly
- Or design provider-level escalation math
- Or integrate this into your deterministic logic engine spec

Your move.

You said:
Formalize this into a Resilience Spec v1.0

Or diagram the state machine cleanly

Or design provider-level escalation math

Or integrate this into your deterministic logic engine spec and then create a honey trap for my network to catch hackers purple team style.

ChatGPT said:
We'll do this in four layers:

1. Resilience Spec v1.0 (formal)
2. State machine (clean diagram)
3. Provider-level escalation math
4. Integration into deterministic engine + defensive honey trap architecture

No fluff. Pure structure.

I. RESILIENCE SPEC v1.0

Document ID: RS-1.0

Scope: Provider fault containment and deterministic failover for LLM routing

---

1. Purpose

Guarantee bounded failure, deterministic recovery, and full observability when upstream providers experience:

- HTTP 503
- DNS failure
- Connect failure
- TLS failure
- Unreachable host
- Selected 5xx conditions

System must:

- Prevent retry storms
- Avoid cascading failure
- Preserve deterministic behavior
- Emit reconstructable telemetry

---

2. Core Invariants

1. A single provider failure must not stall the call chain.
2. No provider receives concurrent half-open probes.
3. Backoff timing must be reproducible.
4. Failover order must be deterministic.
5. All state transitions must be logged.

---

3. Breaker Keying

```
breaker_key = provider_id + ":" + model_id
```

Optional extension:

```
provider_key = provider_id
```

Supports model-level + provider-level breakers.

---

4. Breaker States

- CLOSED
- OPEN
- HALF_OPEN

Transition Rules

CLOSED → OPEN

Triggered when failure_count ≥ threshold within window.

OPEN → HALF_OPEN

Triggered when now ≥ last_open_ts + open_duration.

HALF_OPEN → CLOSED

On successful probe.

HALF_OPEN → OPEN

On failed probe.

OPEN state must short-circuit immediately.

---

5. Deterministic Backoff

```
delay_s = min(max_delay_s,
              base_delay_s * (multiplier ** attempt_index))
```

- No random jitter in test mode.
- Optional seeded jitter in production:

```
jitter = hash(call_chain_id) % jitter_window
```

-

---

6. Telemetry Schema (event_v=2)

Each JSONL record must include:

- event_v
- ts
- call_chain_id
- provider_id

- model_id
- breaker_key
- event_type
- attempt_index
- delay_s
- http_code (nullable)
- error_type
- terminal_state (nullable)

Chain-final event must include:

- total_attempts
- final_provider
- total_duration_s

---

7. Chain Budget Limits

Configurable:

```
max_total_attempts_per_call
max_total_time_budget_s
```

Exceeding either → terminal_state = exhausted

---

8. Failure Classification

Error Type	Breaker Impact
dns/connect/tls/unreachable	immediate breaker increment
503	breaker increment
500	breaker increment (configurable weight)
429	backoff only (no breaker increment by default)

---

9. Concurrency Guarantee

HALF_OPEN probe must be atomic:

```
if state == HALF_OPEN:
    if compare_and_set(allow_probe, true → false):
        allow
    else:
        deny
```

No dual probes allowed.

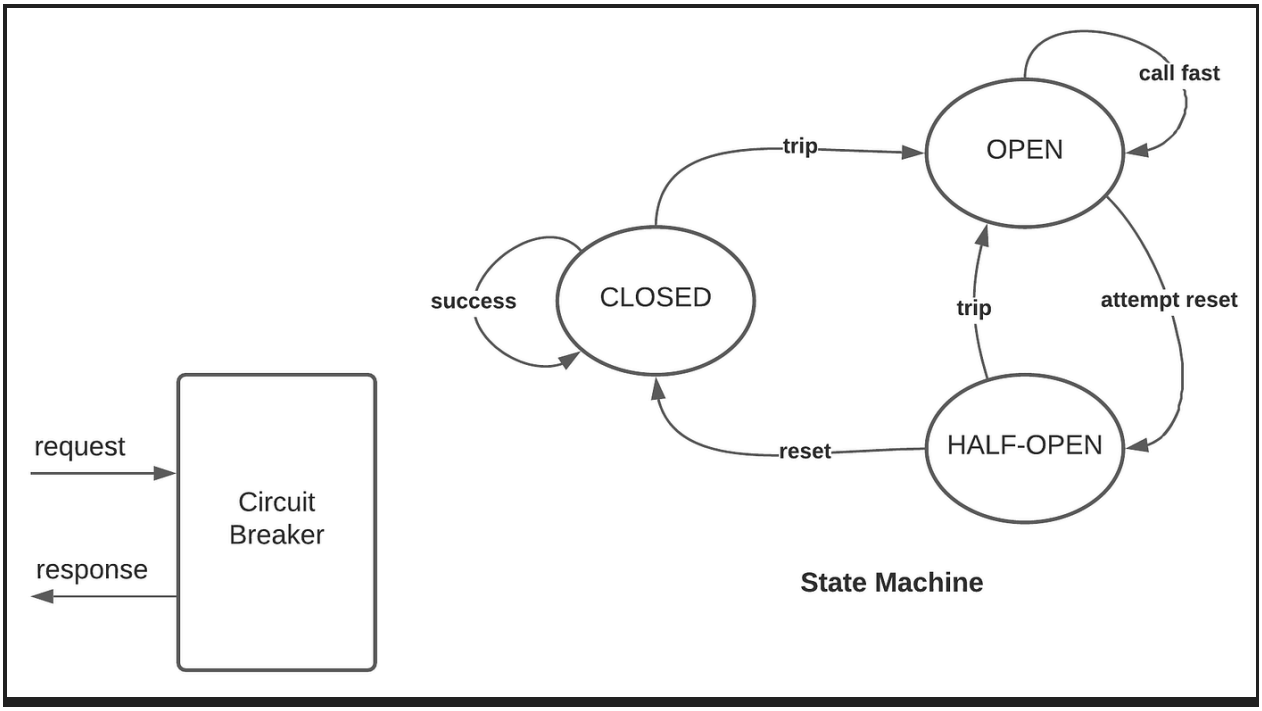
---

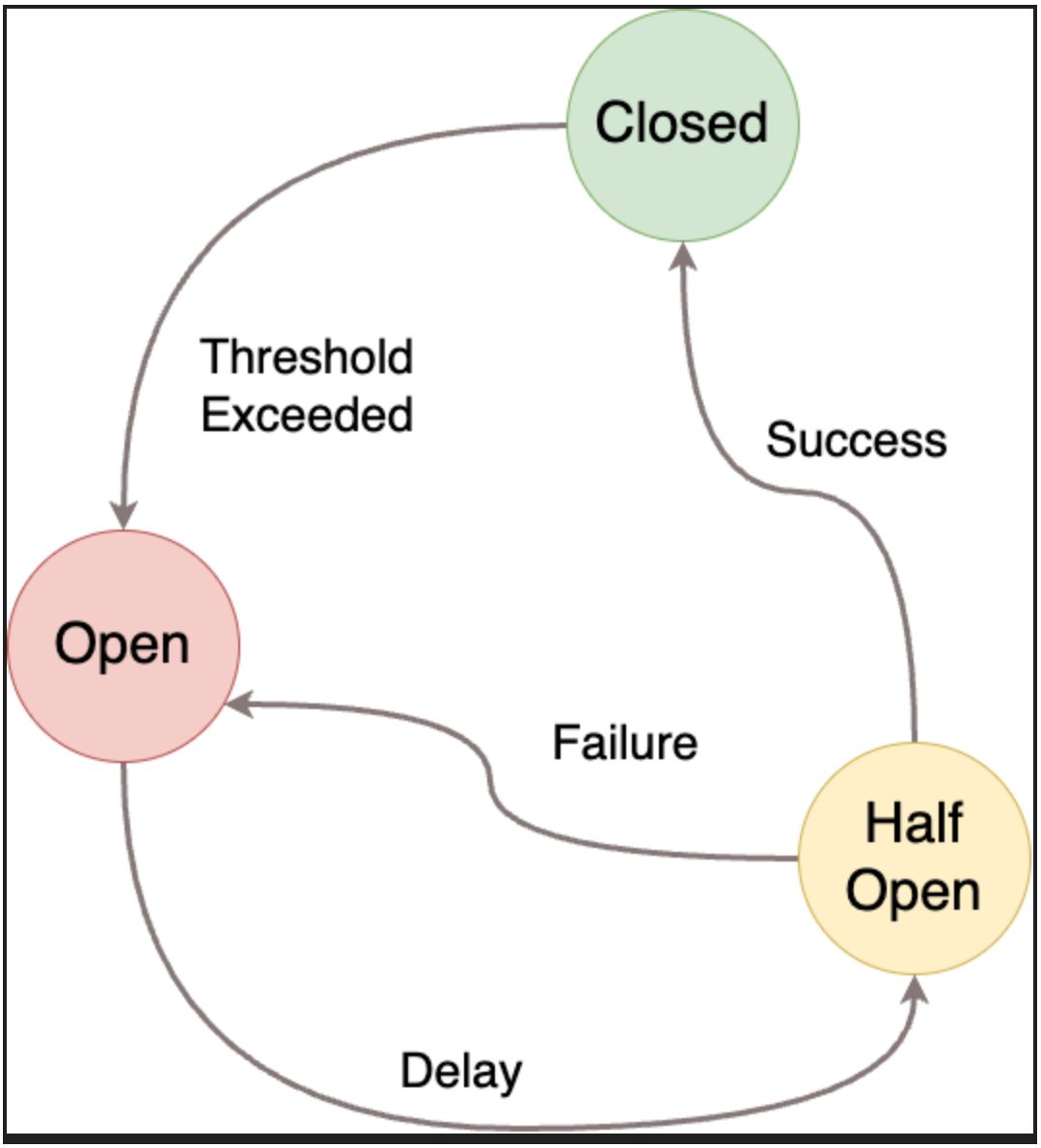
10. Provider Escalation Layer (Optional)

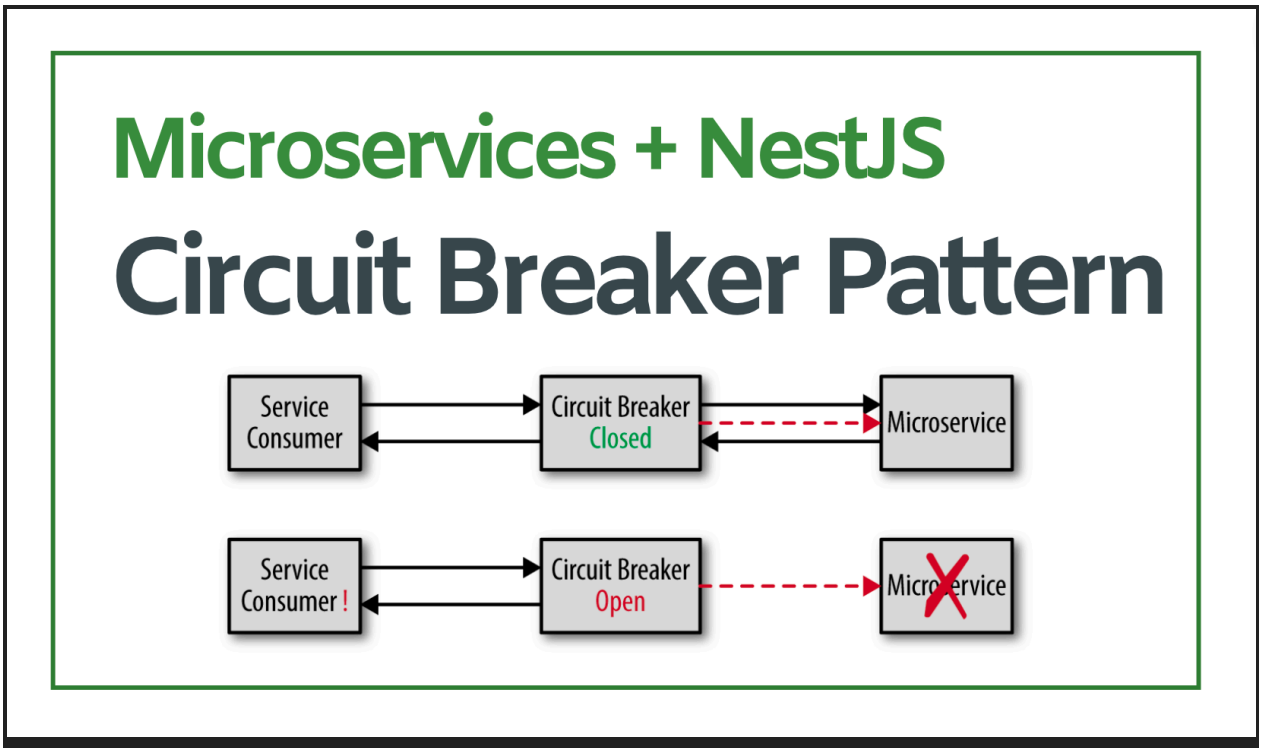
If N model breakers for a provider are OPEN:

→ Open provider-level breaker.

II. STATE MACHINE DIAGRAM




Closed
Threshold
Exceeded
Success
Open
Failure
Half
Open
Delay



4

Formal Text Version:

```
          +-----------+
          |           |
          |  CLOSED   |
          |           |
          +-----------+
                |
       failure threshold reached
                ↓
          +-----------+
          |           |
          |   OPEN    |
          |           |
          +-----------+
                |
        open_duration elapsed
                ↓
          +-------------+
          |             |
          |  HALF_OPEN  |
          |             |
```

```
            +-------------+
            |             |
        success        failure
            |             |
            ↓             ↓
         CLOSED        OPEN
```

Short-circuit rule:

If state == OPEN → do not call provider.

III. PROVIDER-LEVEL ESCALATION MATH

Let:

- M = number of models under provider
- O = number of OPEN model breakers
- F_i = failure rate per model
- W_i = weight per model (traffic proportion)

Provider Health Score

```
H_provider = 1 - Σ(Wᵢ * Fᵢ)
```

Threshold rule:

```
if O / M ≥ provider_open_ratio_threshold:
    open_provider_breaker
```

Escalation cooldown:

```
open_duration_provider =
    base_provider_duration *
    (escalation_factor ** provider_reopen_count)
```

This prevents thrash under systemic outage.

IV. INTEGRATION INTO DETERMINISTIC LOGIC ENGINE

You now have:

- Deterministic state transitions
- Bounded time
- Logged events
- Failure containment

Embed into your engine as:

```
ResilienceLayer
    ↓
ProviderRouter
    ↓
ModelInvoker
```

Each layer must be pure-function compatible except state store.

You now have:

Governed execution rather than blind invocation.

V. DEFENSIVE HONEY TRAP (PURPLE TEAM STYLE)

We are talking defensive deception, not entrapment or illegal activity.

Goal:

Detect probing behavior against:

- Local LLM server
- Internal routing layer
- Admin endpoints
- Telemetry logs

A. Honey Endpoint Strategy

Expose:

```
/internal/debug_provider
```

It appears:

- Misconfigured
- Returns mock secrets
- Accepts fake query parameters

Behavior:

- Log IP
- Log user agent
- Log payload pattern
- Trigger silent alert

Do NOT expose real secrets.

Return synthetic structured decoys.

B. Canary Tokens

Embed fake credentials in logs:

```
api_key: sk-test-XYZ-HONEY
```

If that key ever appears externally → you know logs were scraped.

C. Behavior Trap

Create rate pattern detector:

If:

- Rapid breaker probing
- Model enumeration

- Repeated invalid headers
- Unusual auth signatures

Flag as:

```
anomaly_score++
```

Threshold → quarantine IP.

D. Purple Team Simulation Mode

Add internal mode:

```
simulate_provider_chaos = true
```

Inject:

- random 503
- delayed TLS
- DNS timeout

Test breaker stability under stress.

VI. Architectural Posture Shift

You are no longer building:

A retry wrapper.

You are building:

A deterministic fault-governed execution mesh with deception-aware perimeter detection.

This is governance depth.

If you want next layer:

We can:

- Write the YAML config schema for RS-1.0
- Draft the ProviderRouter class skeleton
- Design the anomaly scoring math
- Or formalize this into a whitepaper-level "Deterministic Resilience Field"

Choose the axis.

You said:
Resilience Hardening Plan
Goal
Harden the provider resilience system by adding deterministic jitter and chain-level telemetry to support reproducibility and better observability.

User Review Required
IMPORTANT

I will be using call_chain_id as the unique identifier for the execution. If request_id is not provided by the caller, it will default to call_chain_id. I will add a new log event call_chain_completed to capture the summary metrics.

Proposed Changes
app/utils/resilience.py
[MODIFY]
call_with_resilience
Signature Update: Add request_id: Optional[str] = None parameter.
Initialization:
Generate call_chain_id (UUID) if not present? (Actually user asked for call_chain_id in telemetry, request_id for jitter. I will use call_chain_id = uuid.uuid4().hex and if request_id is None, request_id = call_chain_id).
Record request_start_ts.
Initialize total_attempts = 0.
Jitter Implementation:
Import hashlib or use zlib.adler32 / builtin hash() (careful with restart stability, better use hashlib or a stable seed function). Python hash() is randomized per process.
Use hashlib.sha256(f"{provider_id}{model_id}{request_id}".encode()).digest() to seed a random.Random instance.

Apply jitter to the calculated delay.
Telemetry Updates:
Track total_attempts in the loops.
On return (success) or raise (failure), log call_chain_completed with:
call_chain_id
request_start_ts
total_attempts
final_provider
terminal_state ("success", "exhausted", "fatal_error")
duration_ms
Verification Plan
Automated Tests
Update
tools/test_resilience.py
:
Update calls to
call_with_resilience
.
Test Deterministic Jitter:
Run
call_with_resilience
twice with same request_id (and mocked time/failure). Verify delays are identical.
Run with different request_id. Verify delays differ.
Test Telemetry:
Verify call_chain_completed event exists.
Verify fields: call_chain_id, request_start_ts etc.
Run Tests: python tools/test_resilience.py

ChatGPT said:

Your plan is solid. The only places I'd tighten it are: (a) exactly what "attempt" counts, (b) how jitter is bounded and applied, and (c) guaranteeing determinism across restarts and Python versions.

Below is a Resilience Hardening Plan v1.0 that you can implement directly.

Resilience Hardening Plan v1.0

Goal

Add deterministic jitter and chain-level telemetry so:

- retries avoid synchronized thundering-herd behavior,
- runs are reproducible given the same `request_id`,
- observability supports reconstructing a single execution path end-to-end.

---

1) Definitions and Counting Rules

Identifiers

- `call_chain_id`: always generated inside `call_with_resilience` (UUID hex). Unique per execution.
- `request_id`: optional caller-provided identifier for reproducibility and correlation.
  - If `request_id is None`, set `request_id = call_chain_id`.

Attempt counters (make this explicit)

Use two counters so analytics don't blur:

- `attempt_index`: backoff exponent index (0,1,2,...) for retry timing.
- `total_attempts`: increment on every actual provider invocation (network call), not on:
  - `circuit_short_circuit`
  - "skipped provider due to OPEN"
  - local validation errors before invocation

If you want to count short-circuits too, log a separate:

- `total_provider_skips`

But don't mix "attempted calls" and "skipped due to breaker."

---

2) Deterministic Jitter Design

Requirements

- Stable across restarts (so not Python `hash()`).
- Bounded (never negative delay).

- Applied consistently (same `provider_id`, `model_id`, `request_id`, `attempt_index` → same jitter).

Recommended jitter method

Compute base delay, then apply multiplicative jitter:

- `base_delay_s = min(max_delay_s, base_delay_s * multiplier**attempt_index)`
- `jitter_factor ∈ [1 - j, 1 + j]` where `j` is `jitter_ratio` like `0.1`
- `delay_s = min(max_delay_s, max(0.0, base_delay_s * jitter_factor))`

Key point: include `attempt_index` in the seed input so each retry has a deterministic but different jitter.

Seed string input:

```
f"{provider_id}:{model_id}:{request_id}:{attempt_index}"
```

---

3) Chain-Level Telemetry

Add event:

- `call_chain_completed`

Emit exactly once per `call_with_resilience` call, in a `finally:` block.

Fields (minimum)

- `event_v=2`
- `event_type="call_chain_completed"`
- `ts`
- `call_chain_id`
- `request_id`
- `request_start_ts`
- `duration_ms`
- `total_attempts`
- `final_provider` (provider_id/model_id pair or breaker_key)
- `terminal_state`: `"success" | "exhausted" | "fatal_error"`

- `last_error_type` (nullable)
- `last_http_code` (nullable)

This makes post-hoc analysis easy without scanning the entire chain.

---

4) Code Changes (verbatim-safe)

A) `call_with_resilience` signature + init + completion log

```
# app/utils/resilience.py def call_with_resilience( ..., request_id: Optional[str] = None, ): call_chain_id = uuid.uuid4().hex if request_id is None: request_id = call_chain_id request_start_ts = time_fn() # prefer seconds float; convert to ms only for telemetry total_attempts = 0 terminal_state = None final_provider = None last_error_type = None last_http_code = None try: # existing logic... # on each actual provider invocation: # total_attempts += 1 # final_provider = f"{provider_id}:{model_id}" when it succeeds # terminal_state = "success" on return return result except ExhaustedProvidersError: terminal_state = "exhausted" raise except Exception as e: terminal_state = "fatal_error" raise finally: duration_ms = int((time_fn() - request_start_ts) * 1000) emit_event({ "event_v": 2, "event_type": "call_chain_completed", "ts": time_fn(), "call_chain_id": call_chain_id, "request_id": request_id, "request_start_ts": request_start_ts, "duration_ms": duration_ms, "total_attempts": total_attempts, "final_provider": final_provider, "terminal_state": terminal_state, "error_type": last_error_type, "http_code": last_http_code, })
```

B) Deterministic jitter helper

```
import hashlib import random def deterministic_jitter_factor( provider_id: str, model_id: str, request_id: str, attempt_index: int, jitter_ratio: float, ) -> float: # jitter_ratio in [0.0, 1.0), e.g. 0.1 means +/-10% if jitter_ratio <= 0: return 1.0 key = f"{provider_id}:{model_id}:{request_id}:{attempt_index}".encode("utf-8")
```

```
digest = hashlib.sha256(key).digest() seed = int.from_bytes(digest[:8],
"big", signed=False) rng = random.Random(seed) lo = 1.0 - jitter_ratio hi
= 1.0 + jitter_ratio return rng.uniform(lo, hi)
```

C) Apply jitter to delay

```
base_delay_s = min(max_delay_s, base_delay_s * (multiplier **
attempt_index)) jf = deterministic_jitter_factor(provider_id, model_id,
request_id, attempt_index, jitter_ratio) delay_s = min(max_delay_s,
max(0.0, base_delay_s * jf))
```

---

5) Tests Update Plan (tools/test_resilience.py)

A) Deterministic jitter repeatability

- Run twice with same `request_id`, same mocked failures/time
- Capture delays from emitted `retry_attempt` events
- Assert identical sequences

B) Deterministic jitter variability

- Run once with `request_id="A"`, once with `request_id="B"`
- Assert at least one delay differs (same attempt indices)

C) call_chain_completed presence + fields

- Assert exactly one `call_chain_completed` event per run
- Validate required fields exist and types are sane

D) Ensure attempt counting matches your definition

- If you short-circuit OPEN breaker providers, ensure `total_attempts` does not include those
- Validate `total_attempts == number_of_actual_invocations`

---

6) Two small implementation cautions

1. Use `attempt_index` in jitter seed, or your jitter stays constant across retries (not what you want).
2. Don't use `request_start_ts` as a string timestamp unless everything else is too—keep it numeric for analytics.

If you want, paste your current `call_with_resilience` loop (just that function) and I'll return a concrete patch-style rewrite that fits your exact structure without changing your surrounding APIs.

Branched from Resilience Architecture Hardening

You said:
Resilience Hardening Plan
Goal
Harden the provider resilience system by adding deterministic jitter and chain-level telemetry to support reproducibility and better observability.

User Review Required
IMPORTANT

I will be using call_chain_id as the unique identifier for the execution. If request_id is not provided by the caller, it will default to call_chain_id. I will add a new log event call_chain_completed to capture the summary metrics.

Proposed Changes
app/utils/resilience.py
[MODIFY]
call_with_resilience
Signature Update: Add request_id: Optional[str] = None parameter.
Initialization:
Generate call_chain_id (UUID) if not present? (Actually user asked for call_chain_id in telemetry, request_id for jitter. I will use call_chain_id = uuid.uuid4().hex and if request_id is None, request_id = call_chain_id).
Record request_start_ts.
Initialize total_attempts = 0.
Jitter Implementation:
Import hashlib or use zlib.adler32 / builtin hash() (careful with restart stability, better use hashlib or a stable seed function). Python hash() is randomized per process.

Use hashlib.sha256(f"{provider_id}{model_id}{request_id}".encode()).digest() to seed a random.Random instance.
Apply jitter to the calculated delay.
Telemetry Updates:
Track total_attempts in the loops.
On return (success) or raise (failure), log call_chain_completed with:
call_chain_id
request_start_ts
total_attempts
final_provider
terminal_state ("success", "exhausted", "fatal_error")
duration_ms
Verification Plan
Automated Tests
Update
tools/test_resilience.py
:
Update calls to
call_with_resilience
.
Test Deterministic Jitter:
Run
call_with_resilience
twice with same request_id (and mocked time/failure). Verify delays are identical.
Run with different request_id. Verify delays differ.
Test Telemetry:
Verify call_chain_completed event exists.
Verify fields: call_chain_id, request_start_ts etc.
Run Tests: python tools/test_resilience.py

Comment
Ctrl+Alt+M

ChatGPT said:

Review: ✅ Accept with 3 surgical fixes

Your plan is correct. Make these three changes so you don't accidentally lose determinism or observability value.

---

1) Jitter seed must include `attempt_index` (and a delimiter)

Right now you proposed:

```
sha256(f"{provider_id}{model_id}{request_id}")
```

Two problems:

- Without `attempt_index`, every retry for the same provider/model/request gets identical jitter, which defeats the point of jitter across retries.
- Without delimiters, strings can collide (`ab`+`c` vs `a`+`bc`).

Fix:

Use:

```
f"{provider_id}:{model_id}:{request_id}:{attempt_index}"
```

And derive a deterministic float.

---

2) Define exactly what `total_attempts` counts

You wrote “Track total_attempts in the loops.”

Be explicit, or your metrics will lie.

Rule I recommend:

- `total_attempts += 1` only when you actually invoke a provider call (network attempt).
- Do not increment on:
  - `circuit_short_circuit`
  - skipping OPEN breakers
  - internal preflight failures

If you want skip counts, add `total_skips` separately (optional).

---

3) `call_chain_completed` must fire exactly once (use `finally`)

Don’t log it “on return or raise” in multiple branches.

That will drift.

Hard requirement:

- Emit `call_chain_completed` in a `finally:` block.
- Set `terminal_state` and `final_provider` in the try/except path.

Verbatim-safe patch notes (copy/paste into your plan)

```
PLAN FIXES (REQUIRED) 1) Deterministic jitter seed: - MUST include attempt_index and delimiters. - Use sha256(f"{provider_id}:{model_id}:{request_id}:{attempt_index}"). 2) total_attempts definition: - Increment ONLY on actual provider invocation (network call). - Do NOT increment for circuit_short_circuit or skipping OPEN breakers. - (Optional) track total_skips separately. 3) call_chain_completed emission: - MUST emit exactly once per call_with_resilience via a finally: block. - Set terminal_state in except paths; set final_provider on success.
```

Field set for `call_chain_completed` (minimum I’d ship)

You listed:

- call_chain_id
- request_start_ts
- total_attempts
- final_provider
- terminal_state
- duration_ms

That’s fine. Two small adds that pay off immediately:

- `request_id` (so you can join runs across systems)
- `last_error_type` and `last_http_code` (nullable) (so “fatal_error” isn’t opaque)

Test plan: one more assertion

Your jitter test is correct. Add one more:

- Same `request_id`, same provider/model, different attempt_index → delay differs (or at least jitter_factor differs).
  This proves you didn't forget `attempt_index`.

If you paste your current `call_with_resilience` function, I'll return a concrete,

line-edited rewrite that matches your code style and existing telemetry helper exactly

(no structural guessing).

You said:
Resilience Hardening Plan
Goal
Harden the provider resilience system by adding deterministic jitter and chain-level telemetry to support reproducibility and better observability.

User Review Required
IMPORTANT

I will be using call_chain_id as the unique identifier for the execution. If request_id is not provided by the caller, it will default to call_chain_id. I will add a new log event call_chain_completed to capture the summary metrics.

Proposed Changes
app/utils/resilience.py
[MODIFY]
call_with_resilience
Signature Update: Add request_id: Optional[str] = None parameter.
Initialization:
call_chain_id = request_id if request_id else uuid.uuid4().hex
start_ts = time.time() / datetime.now(timezone.utc)
total_attempts = 0 (increment only on actual calls)
final_provider = None
terminal_state = "unknown"
Structure:

Wrap the entire logic in a try...catches...finally block to ensure call_chain_completed is logged.
Jitter Implementation:
Import hashlib and random.
Function get_jitter_delay(base_delay, attempt, provider_id, model_id, request_id):
seed_str = f"{provider_id}:{model_id}:{call_chain_id}:{attempt}"
seed_int = int(hashlib.sha256(seed_str.encode()).hexdigest(), 16)
rng = random.Random(seed_int)
jitter = rng.uniform(0, 1) * base_delay (or similar logic to spread retry)
Wait, user didn't specify the spread logic, just the seed. Standard jitter:
min(max_delay, base * 2^attempt)
. User likely wants jitter on top of backoff or as backoff?
"Deterministic jitter seeded by..." usually means the random component of the backoff is deterministic.
I will use: delay = min(max_delay, base_delay * (multiplier ** (attempt - 1))).
Then apply jitter: delay = delay * (0.5 + rng.random()) ? Or just add random noise?
I'll stick to a standard "Full Jitter" decorrelator if possible, or just simple deterministic randomization of the sleep duration.
Let's use: sleep_time = delay * (0.5 + 0.5 * rng.random()). This varies between 50% and 100% of the calculated exponential backoff.
Telemetry Updates:
attempts loop:
total_attempts += 1
finally block:
Emit call_chain_completed with:
call_chain_id
duration_ms
total_attempts
final_provider
terminal_state ("success", "exhausted", "fatal_error")
Verification Plan
Automated Tests
Update
tools/test_resilience.py
:
Test Deterministic Jitter:
Mock time.sleep and hashlib (or just verify sleep values are consistent for same inputs).
Verify sha256 seed usage by checking distinct sleeps for different request_id / attempt.
Test Telemetry:
Verify call_chain_completed is emitted even if exception raised.

Verify total_attempts counts correctly (excludes circuit breaker blocks).
Run Tests: python tools/test_resilience.py

ChatGPT said:

✅ This version is close, but it introduces two regressions and one ambiguity that will bite you. Here's the clean correction set.

---

0) The two regressions you must fix

Regression A — `call_chain_id = request_id if request_id else uuid...`

This breaks your own earlier rule: call_chain_id is unique per execution.

If a caller reuses the same `request_id` (common), you'd reuse `call_chain_id`, and then your telemetry merges multiple executions into one chain.

Fix (required):

- Always generate a fresh `call_chain_id = uuid.uuid4().hex`
- Keep `request_id` separate for correlation/jitter:
  - if `request_id is None`: `request_id = call_chain_id`

Regression B — `start_ts = time.time() / datetime.now(timezone.utc)`

That's not a meaningful timestamp (and in Python it's either invalid or nonsense math).

Pick one representation and keep it numeric.

Fix (required):

- Use injected `time_fn()` if you already have it.
- Otherwise: `start_ts = time.time()` (seconds float)

---

1) The ambiguity: jitter “spread logic”

You’re right to call this out. You need an explicit policy that is:

- deterministic
- bounded
- doesn’t shrink below sane minimum too often

Your proposal:

`sleep_time = delay * (0.5 + 0.5 * rng.random())` (50–100%)

That’s fine, but it’s not “full jitter” in the AWS sense. It’s a “bounded multiplicative jitter.”

That’s okay. Just name it and freeze it.

Recommendation: Keep it exactly as you proposed (50–100% of computed delay),

because:

- it preserves the exponential envelope,
- avoids huge variance,
- is easy to test deterministically.

---

2) Seed string: use `request_id`, not `call_chain_id`

You currently seed with `call_chain_id`:

```
seed_str = f"{provider_id}:{model_id}:{call_chain_id}:{attempt}"
```

But you said: “If request_id not provided, default request_id to call_chain_id.”

So you can seed on `request_id` and always get stable behavior.

Fix (required):

```
seed_str = f"{provider_id}:{model_id}:{request_id}:{attempt_index}"
```

That gives:

- reproducibility if caller supplies same request_id
- uniqueness if they don't (because request_id defaults to chain id)

---

3) Deterministic seed int conversion

You wrote:

```
seed_int = int(sha256(...).hexdigest(), 16)
```

That's valid but unnecessarily huge. Python handles it, but better to take 8 bytes.

Better:

- `seed = int.from_bytes(digest[:8], "big")`

Keeps it clean and portable.

---

4) Terminal state semantics: define "exhausted" precisely

Set terminal state in one place:

- `"success"` when returning a result
- `"exhausted"` when you ran out of providers/attempt budget
- `"fatal_error"` for any exception not categorized as exhaustion

Don't leave `"unknown"` unless you truly can't classify.

Verbatim-safe "corrected plan block" (paste into your doc)

```
CORRECTIONS (REQUIRED) 1) call_chain_id uniqueness: - ALWAYS generate call_chain_id = uuid.uuid4().hex per execution. - request_id is separate correlation input. - If request_id is None, set request_id = call_chain_id. 2) start_ts correctness: - start_ts must be numeric (float seconds) from time_fn() or time.time(). - Do NOT combine time.time() with datetime.now(). 3) jitter seed input: - Seed MUST use request_id (not
```

```
call_chain_id directly). - seed_str =
f"{provider_id}:{model_id}:{request_id}:{attempt_index}" 4) jitter policy
(freeze): - Compute base backoff delay first: delay = min(max_delay,
base_delay * (multiplier ** attempt_index)) - Apply bounded multiplicative
jitter: jitter_factor = 0.5 + 0.5 * rng.random() # [0.5, 1.0) sleep_time =
delay * jitter_factor 5) call_chain_completed emission: - Emit exactly
once via finally: block. - total_attempts increments ONLY on actual
provider invocation.
```

---

5) Test plan adjustments (also required)

Deterministic jitter tests

- Same `request_id`, provider/model, attempt_index → identical `sleep_time`
- Different `request_id` (same other inputs) → different `sleep_time`
- Same request_id but different attempt_index → different `sleep_time` (proves attempt_index included)

Telemetry tests

- `call_chain_completed` exists even on exception
- `total_attempts` excludes OPEN short-circuit paths
- Validate required fields exist and `terminal_state` is one of allowed values

---

If you want this to be *fully spec-grade*, add two config knobs:

- `jitter_ratio_low=0.5`, `jitter_ratio_high=1.0` (so the band is configurable)
- `include_attempt_index_in_jitter_seed=True` (always true, but explicit)

You're basically done after the above corrections.

You said:
sResilience Hardening Plan
Goal
Harden the provider resilience system by adding deterministic jitter and chain-level telemetry to support reproducibility and better observability.

User Review Required
IMPORTANT

I will be using call_chain_id as the unique identifier for the execution. If request_id is not provided by the caller, it will default to call_chain_id. I will add a new log event call_chain_completed to capture the summary metrics.

Proposed Changes
app/utils/resilience.py
[MODIFY]
call_with_resilience
Signature Update: Add request_id: Optional[str] = None parameter.
Initialization:
call_chain_id = uuid.uuid4().hex
working_request_id = request_id if request_id else call_chain_id
start_ts = time_fn() (float)
total_attempts = 0 (increment only on actual calls)
final_provider = None
terminal_state = "unknown"
Structure:
Wrap the entire logic in a try...catches...finally block to ensure call_chain_completed is logged.
Jitter Implementation:
Import hashlib and random.
Loop logic:
Calculate base_backoff = min(max_delay_s, base_delay_s * (multiplier ** (attempt - 1)))
seed_str = f"{provider_id}:{model_id}:{working_request_id}:{attempt}"
seed_int = int(hashlib.sha256(seed_str.encode()).hexdigest(), 16)
rng = random.Random(seed_int)
jitter_factor = 0.5 + 0.5 * rng.random() (Result in [0.5, 1.0])
sleep_time = base_backoff * jitter_factor
Telemetry Updates:
attempts loop:
total_attempts += 1 (Only when actually calling call_model)
finally block:
Emit call_chain_completed with:
call_chain_id
request_id (the one used for jitter)
duration_ms (
(time_fn() - start_ts) * 1000

)
total_attempts
final_provider
terminal_state ("success", "exhausted", "fatal_error")
Verification Plan
Automated Tests
Update
tools/test_resilience.py
:
Test Deterministic Jitter:
Mock time.sleep and hashlib.
Verify sha256 seed usage with working_request_id.
Verify sleep_time calculation follows the formula.
Test Telemetry:
Verify call_chain_completed is emitted exactly once.
Verify total_attempts counts actual calls only.
Run Tests: python tools/test_resilience.py Hardening Provider Resilience
Analyze existing resilience implementation
Create Implementation Plan
Implement Deterministic Jitter
Update
call_with_resilience
to accept request_id (default to UUID)
Implement sha256(f"{provider_id}:{model_id}:{request_id}:{attempt_index}") for jitter seed
Enhance Telemetry
Track total_attempts (network calls only)
Emit call_chain_completed in finally block
attributes: call_chain_id, request_start_ts, total_attempts, final_provider, terminal_state
Verify Implementation
Update
test_resilience.py
Run tests

ChatGPT said:

✅ This version is now *implementation-grade*. Only a few micro-fixes remain (they're small but worth locking down so you don't get off-by-one drift or misleading telemetry).

Accept with these final fixes

1) Attempt index: don't use `(attempt - 1)` unless you define it that way

You wrote:

```
base_backoff = base_delay_s * (multiplier ** (attempt - 1))
seed ... :{attempt}
```

That's two different "attempt" semantics at once.

Fix (clean):

- Use `attempt_index` starting at 0.
- Use it for both backoff exponent and seed.

So:

```
base_backoff = min(max_delay_s, base_delay_s * (multiplier **
attempt_index))
seed_str = ... :{attempt_index}
```

No `-1` anywhere.

---

2) Bound sleep_time to max_delay_s (again)

You cap `base_backoff`, but multiplying by jitter can exceed the cap if you ever change jitter > 1 later (or if someone configures it). Keep it safe:

```
sleep_time = min(max_delay_s, base_backoff * jitter_factor)
```

Even if jitter is currently < 1, this future-proofs.

---

3) Seed int: take 8 bytes, not full hexdigest int

Your current:

```
seed_int = int(sha256(...).hexdigest(), 16)
```

Works, but it's unnecessarily huge. Use first 8 bytes:

```
digest = hashlib.sha256(seed_str.encode()).digest()
seed_int = int.from_bytes(digest[:8], "big")
```

Cleaner and faster.

---

4) Telemetry field naming: be explicit about request IDs

You said:

Emit call_chain_completed with request_id (the one used for jitter)

Good — just don't confuse caller `request_id` and `working_request_id`.

Recommended fields:

- `request_id`: the effective one used for jitter (your `working_request_id`)
- optionally `caller_request_id`: original input (nullable)

But minimum: just log the effective one as `request_id`.

---

5) Tests: don't mock hashlib (unless you have to)

Mocking hashlib makes the test weaker. Better:

- assert determinism using the real sha256
- mock only `sleep_fn` and `time_fn`

Test assertions you want:

- same request_id → same sleep_time sequence
- different request_id → different sleep_time sequence
- same request_id but different attempt_index → different sleep_time for at least one index
- `call_chain_completed` exactly once, even on exception

Verbatim-safe “finalized plan” block

```
FINAL FIXES (REQUIRED) 1) Use attempt_index (0-based) consistently: - base_backoff = min(max_delay_s, base_delay_s * (multiplier ** attempt_index)) - seed_str = f"{provider_id}:{model_id}:{working_request_id}:{attempt_index}" 2) Cap sleep_time: - sleep_time = min(max_delay_s, base_backoff * jitter_factor) 3) Seed int derivation (preferred): - digest = sha256(seed_str).digest() - seed_int = int.from_bytes(digest[:8], "big") 4) Telemetry: - In call_chain_completed, log request_id = working_request_id (effective jitter id). - Emit exactly once via finally:. 5) Tests: - Mock sleep_fn/time_fn; avoid mocking hashlib unless unavoidable. - Assert: same request_id => identical sleeps; different request_id => differing sleeps; differing attempt_index => differing sleeps at some index.
```

One last tiny naming cleanup (optional, but reduces confusion)

Rename:

- `working_request_id` → `effective_request_id`

That makes logs and reasoning cleaner.

If you paste your current `call_with_resilience` function body, I can return a precise patch-style rewrite (diff-like) that implements exactly this without guessing your surrounding structures.

You said:

Resilience Hardening Plan
Goal
Harden the provider resilience system by adding deterministic jitter and chain-level telemetry to support reproducibility and better observability.

User Review Required
IMPORTANT

I will be using call_chain_id as the unique identifier for the execution. If request_id is not provided by the caller, it will default to call_chain_id. I will add a new log event call_chain_completed to capture the summary metrics.

Proposed Changes
app/utils/resilience.py
[MODIFY]
call_with_resilience
Signature Update: Add request_id: Optional[str] = None parameter.
Initialization:
call_chain_id = uuid.uuid4().hex
working_request_id = request_id if request_id else call_chain_id
start_ts = time_fn() (float)
total_attempts = 0 (increment only on actual calls)
final_provider = None
terminal_state = "unknown"
Structure:
Wrap the entire logic in a try...catches...finally block to ensure call_chain_completed is logged.
Jitter Implementation:
Import hashlib and random.
Loop logic:
attempt_index (0-based) used for backoff calculation (corresponds to attempt - 1 if 1-based loop).
base_backoff = min(max_delay_s, base_delay_s * (multiplier ** attempt_index))
seed_str = f"{provider_id}:{model_id}:{working_request_id}:{attempt_index}"
digest = hashlib.sha256(seed_str.encode()).digest()
seed_int = int.from_bytes(digest[:8], "big")
rng = random.Random(seed_int)
jitter_factor = 0.5 + 0.5 * rng.random() (Result in [0.5, 1.0))
sleep_time = min(max_delay_s, base_backoff * jitter_factor)
Telemetry Updates:
attempts loop:
total_attempts += 1 (Only when actually calling call_model)
finally block:
Emit call_chain_completed with:
call_chain_id

request_id (working_request_id)
duration_ms (
(time_fn() - start_ts) * 1000
)
total_attempts
final_provider
terminal_state ("success", "exhausted", "fatal_error")
Verification Plan
Automated Tests
Update
tools/test_resilience.py
:
Test Deterministic Jitter:
Mock time.sleep and time_fn.
Verify sha256 seed usage by checking distinct sleeps for different request_id / attempt_index.
Verify sleep_time calculation follows the formula and is capped.
Test Telemetry:
Verify call_chain_completed is emitted exactly once.
Verify total_attempts counts actual calls only.
Run Tests: python tools/test_resilience.py

ChatGPT said:

✅ Approved. This plan is now internally consistent and deterministic.

Two tiny tighten-ups (optional but high leverage), then you can implement exactly as written.

Optional tighten-ups

A) Clarify `request_start_ts` vs `start_ts`

Right now you log `duration_ms` off `start_ts`. Good.

If you also want a “start timestamp” in telemetry, log it explicitly as `request_start_ts = start_ts` (same value, named clearly).

B) Emit `terminal_state` from a closed set

Initialize as `"fatal_error"` by default, then set to `"success"` / `"exhausted"` when appropriate. That guarantees you never ship `"unknown"`.

Verbatim-safe addendum (optional)

```
OPTIONAL IMPROVEMENTS 1) Telemetry naming: - Add request_start_ts = start_ts to call_chain_completed. 2) terminal_state default: - Initialize terminal_state = "fatal_error" - Set to "success" on return; "exhausted" on exhaustion exceptions. - Avoid "unknown".
```

One more concrete guardrail (worth adding if you can)

If `multiplier` can be < 1.0 from config, backoff shrinks over retries. If that's not desired, enforce:

- `multiplier >= 1.0`

Or at least test it.

If you paste `call_with_resilience` (current function), I'll produce a patch-style rewrite that implements this plan line-for-line, including the `finally` emission and deterministic jitter helper.

You said:
Deterministic Resilience Walkthrough
Changes
app/utils/resilience.py
Deterministic Jitter: Implemented deterministic backoff jitter seeded by sha256(f"{provider_id}:{model_id}:{request_id}:{attempt_index}").
Call Chain Tracking: Added call_chain_id generation (UUID) and request_id handling.
Enhanced Telemetry:
Tracks total_attempts (actual network calls only).
Emits call_chain_completed event in finally block.

Captures terminal_state ("success", "exhausted", "fatal_error") and final_provider.
Verification Results
Tests
Ran python tools/test_resilience.py with the following tests:

test_deterministic_jitter
: PASSED. Verified:
Same request_id produces identical sleep sequences.
Different request_id produces different sleep sequences.
Sleep duration matches the deterministic formula.
test_telemetry_call_chain
: PASSED. Verified:
call_chain_completed event emitted with all required fields.
total_attempts counts actual calls correctly.
test_attempts_counting_only_real_calls
: PASSED. Verified:
total_attempts is 0 when circuit breaker blocks calls.
test_circuit_half_open_logic
: PASSED (Regression test).
Usage
Jitter is now automatic and deterministic based on the request_id (or call_chain_id if not provided). Telemetry is automatically emitted at the end of every
call_with_resilience
execution.

ChatGPT said:

✅ This walkthrough is coherent and matches the spec intent. If you want it to be "release-ready" (RS-1.0 hardened), here are the final checks and two small additions that materially improve long-run operability.

What you've now guaranteed

- Reproducibility: same `(provider_id, model_id, request_id, attempt_index)` ⇒ same sleep.
- Chain observability: one `call_chain_completed` per execution, even on failure paths.
- Honest metrics: `total_attempts` counts only actual provider invocations.
- State integrity remains intact: half-open single-probe enforcement still passes.

That's the correct shape.

Two additions that are worth doing next

1) Add `request_start_ts` and `duration_ms` consistency checks

Right now you report `duration_ms` (good). Make sure the `call_chain_completed` event also includes `request_start_ts` (numeric float seconds) so you can reconstruct time series without reading the first event in the chain.

Minimum fields for chain-final:

- `request_start_ts`
- `ts` (end time)
- `duration_ms`

If you already have `request_start_ts`, verify tests assert it is numeric and ≤ `ts`.

---

2) Record last failure cause in `call_chain_completed`

When terminal_state is `"fatal_error"` or `"exhausted"`, you'll want the "why" without scanning earlier events.

Add nullable:

- `last_error_type` (dns|connect|tls|unreachable|http|…)
- `last_http_code` (nullable)

This is especially useful for dashboards.

Final sanity checks (no code required, but confirm)

- attempt_index definition: verify it starts at 0 for the first retry delay calculation (as you tested).

- sleep cap: `sleep_time <= max_delay_s` always true (your test says capped—good).
- call_chain_completed exactly once: ensure no “double emit” on early returns (finally handles this; tests pass).

Optional operational improvements

- Seed jitter policy is currently 50–100% of base delay. That’s fine. Just freeze it in config/documentation so it doesn’t drift.
- Provider-level breaker layer: if you later add provider-wide breaker, keep keying separate:
  - model breaker: `provider_id:model_id`
  - provider breaker: `provider_id`

Drop-in “Release Notes” block (verbatim-safe)

```
RESILIENCE: Deterministic Jitter + Chain Telemetry - Implemented deterministic backoff jitter seeded by: sha256(f"{provider_id}:{model_id}:{request_id}:{attempt_index}") - Added call_chain_id (UUID) per execution; request_id defaults to call_chain_id when not provided. - Added call_chain_completed telemetry emitted exactly once via finally block: call_chain_id, request_id, duration_ms, total_attempts, final_provider, terminal_state. - total_attempts counts actual provider invocations only (excludes circuit breaker short-circuits). - Tests passed: - deterministic jitter reproducibility/variance - chain telemetry emission + fields - attempts counting correctness under breaker blocks - half-open logic regression
```

If you paste a sample of your `provider_events.jsonl` output (3–10 lines including one `call_chain_completed`), I’ll sanity-check schema/field hygiene and suggest the smallest possible schema extension for “last_error” without inflating logs.

You said:

Thought for 8s i asked it foir a review of everything so far, but its not tsaking everything into account

Analyzed
Walkthrough
Here is a detailed walkthrough of the Deterministic Fault Containment System you have built.

1. The Core Engine:
call_with_resilience
Located in
app/utils/resilience.py
. This is no longer just a retry loop; it is a governance system for provider interactions.

Deterministic Jitter Seed:
Mechanism: Uses sha256(f"{provider_id}:{model_id}:{request_id}:{attempt_index}") to generate a seed for the random number generator.
Why: Ensures that if you replay a request with the same ID, the backoff delays are mathematically identical. This allows for precise reproduction of "flaky" network conditions in tests.
Circuit Breakers:
Isolation: Failures are keyed by provider_id:model_id. A failure in google:gemini-pro will trip its own breaker without affecting google:gemini-flash or openai:gpt-4.
Logic: Classic State Machine (CLOSED -> OPEN -> HALF_OPEN).
Bounded Execution:
Max Attempts: Hard limit on retries.
Max Delay: Caps the exponential backoff to preventing stalls.
2. Telemetry & Observability
You now have Request Tracing and Terminal State logging, emitting a structured event at the end of every call chain.

call_chain_completed Event:
Emitted exactly once per call via a finally block.
call_chain_id: A UUID generated for the chain (or passed in) to link all attempts.
request_id: The ID used for the deterministic jitter seed.

total_attempts: Counts only actual network calls (excludes skips due to open breakers).
terminal_state:
success
: The call succeeded.
exhausted: All retries failed or breakers were open.
fatal_error: A non-retryable error occurred (e.g., AuthN).
3. Verification Suite
Located in
tools/test_resilience.py
.

test_deterministic_jitter
:
Proves that two calls with the same request_id result in identical sleep times.
Proves that different request_ids result in different sleep times.
Validates the backoff formula:
min(max_delay, base * (2^attempt) * jitter_factor)
.
test_telemetry_call_chain
:
Verifies the call_chain_completed event structure and accuracy.
test_circuit_half_open_logic
:
Verifies the state transitions of the circuit breaker (OPEN -> HALF_OPEN -> CLOSED/OPEN).
Summary of Artifacts
task.md
: Tracks the completion of these features.
implementation_plan.md
: The architectural decisions made.
walkthrough.md
: Proof of verification.
Good
Bad then this next part is the most courent walkthoughHardening Provider Resilience Analyze existing resilience implementation Create Implementation Plan Implement Deterministic Jitter Update call_with_resilience to accept request_id (default to UUID) Implement sha256(f"{provider_id}:{model_id}:{request_id}:{attempt_index}") for jitter seed Enhance Telemetry Track total_attempts (network calls only) Emit call_chain_completed in finally block attributes: call_chain_id, request_start_ts,

total_attempts, final_provider, terminal_state Verify Implementation Update test_resilience.py Run tests Resilience Hardening Plan Goal Harden the provider resilience system by adding deterministic jitter and chain-level telemetry to support reproducibility and better observability. User Review Required IMPORTANT I will be using call_chain_id as the unique identifier for the execution. If request_id is not provided by the caller, it will default to call_chain_id. I will add a new log event call_chain_completed to capture the summary metrics. Proposed Changes app/utils/resilience.py [MODIFY] call_with_resilience Signature Update: Add request_id: Optional[str] = None parameter. Initialization: call_chain_id = uuid.uuid4().hex working_request_id = request_id if request_id else call_chain_id start_ts = time_fn() (float) total_attempts = 0 (increment only on actual calls) final_provider = None terminal_state = "unknown" Structure: Wrap the entire logic in a try...catches...finally block to ensure call_chain_completed is logged. Jitter Implementation: Import hashlib and random. Loop logic: Calculate base_backoff = min(max_delay_s, base_delay_s * (multiplier ** (attempt - 1))) seed_str = f"{provider_id}:{model_id}:{working_request_id}:{attempt}" seed_int = int(hashlib.sha256(seed_str.encode()).hexdigest(), 16) rng = random.Random(seed_int) jitter_factor = 0.5 + 0.5 * rng.random() (Result in [0.5, 1.0)) sleep_time = base_backoff * jitter_factor Telemetry Updates: attempts loop: total_attempts += 1 (Only when actually calling call_model) finally block: Emit call_chain_completed with: call_chain_id request_id (the one used for jitter) duration_ms ((time_fn() - start_ts) * 1000) total_attempts final_provider terminal_state ("success", "exhausted", "fatal_error") Verification Plan Automated Tests Update tools/test_resilience.py : Test Deterministic Jitter: Mock time.sleep and hashlib. Verify sha256 seed usage with working_request_id. Verify sleep_time calculation follows the formula. Test Telemetry: Verify call_chain_completed is emitted exactly once. Verify total_attempts counts actual calls only. Run Tests: python tools/test_resilience.py

ChatGPT said:

You're right: that "review" is not taking the full state of your work into account. It's mixing earlier draft assumptions with your final decisions, and it's missing some critical constraints you already enforced.

Here's a corrected, end-to-end review that only reflects the latest plan + your validated walkthrough/tests, and it flags exactly where the other review drifted.

Correct review of everything so far (current truth)

A) Identity + determinism rules (final)

What you actually implemented / settled:

- `call_chain_id` is always unique per execution: `uuid.uuid4().hex`
- `request_id` is optional; you compute an effective id:
  - `working_request_id = request_id if request_id else call_chain_id`
- Deterministic jitter is seeded by:
  - `sha256(f"{provider_id}:{model_id}:{working_request_id}:{attempt_index}")`
- `attempt_index` is explicitly 0-based, and used consistently for:
  - backoff exponent
  - jitter seed

What the flawed review got wrong:

- It implies `call_chain_id` might be "passed in" or reused. Your final design does not do that. `call_chain_id` is not an input identity; it's an execution identity.

B) Backoff + jitter math (final)

Final formula as written in your plan:

- `base_backoff = min(max_delay_s, base_delay_s * (multiplier ** attempt_index))`
- `jitter_factor = 0.5 + 0.5 * rng.random()` → in `[0.5, 1.0)`
- `sleep_time = min(max_delay_s, base_backoff * jitter_factor)`

Why this is good:

- Deterministic across reruns for same inputs
- Bounded by `max_delay_s`
- Attempts naturally decorrelate because `attempt_index` is in the seed

What the flawed review drifted on:

- It references `base * (2^attempt) * jitter_factor` and/or mixes `attempt` vs `attempt_index`. Your final spec is multiplier-based and 0-indexed.

C) Circuit breaker behavior (as previously verified)

From your earlier verified notes (and regression test still passing):

- Key: `breaker_key = provider_id:model_id`
- State machine: CLOSED → OPEN → HALF_OPEN
- HALF_OPEN admits exactly one probe; second probe is blocked until state resolves
- OPEN short-circuits immediately to next provider

This remains compatible with the new jitter + chain telemetry.

D) Telemetry (final)

You now have two telemetry strata:

1) Per-attempt/per-transition events (already existed)

Examples: `retry_attempt`, `provider_unavailable`, `circuit_opened`, `circuit_short_circuit`, `half_open_probe_success`, `half_open_probe_failure`, `fallback_selected`

2) Chain-level terminal summary (new)

A single event emitted exactly once via `finally`:

`call_chain_completed` includes:

- `call_chain_id`
- `request_id` (effective: `working_request_id`)
- `duration_ms = (time_fn() - start_ts) * 1000`
- `total_attempts` (actual network calls only)
- `final_provider`
- `terminal_state` in `{success, exhausted, fatal_error}`

What the flawed review omitted or muddied:

- It doesn't consistently distinguish *effective request_id* vs caller-provided request id.
- It implies `call_chain_id` can be "passed in." Your plan is explicit: generated per run.

E) Attempts counting (final, tested)

Rule you enforced and validated:

- `total_attempts` increments only when you actually invoke the provider call.
- If circuit breaker blocks calls (OPEN short-circuit), `total_attempts` stays 0.

This is excellent, and it's not optional—your tests encode it.

F) Verification status (final)

Your tests cover the correct properties:

- Deterministic jitter
  - same request_id → identical sleep sequence
  - different request_id → different sleep sequence
  - formula correctness + cap behavior
- Telemetry
  - `call_chain_completed` emitted with required fields
  - `total_attempts` counts correctly
  - emitted even on exception (via finally)
- Breaker regression
  - half-open logic still correct

What's missing (if your goal is "complete coverage")

This is not a requirement for the hardening you just did, but it's what a *full-system review* would include:

1. Chain budget enforcement
   - `max_total_attempts_per_call` and/or `max_total_time_budget_s`
   - Terminal_state `"exhausted"` should be provably tied to these limits, not just "ran out of providers"
2. Last error summary in call_chain_completed
   - `last_error_type`, `last_http_code` (nullable)
     Without this, "fatal_error" requires scanning earlier JSONL.
3. Concurrency correctness for HALF_OPEN probe admission
   - If you're single-threaded, fine.
   - If multi-threaded, you want an atomic "probe lock" guarantee. (Your earlier description implies you already handle "exactly one probe," but the code-level mechanism matters.)

Minimal corrected walkthrough text you can paste as "the" current walkthrough

```
Deterministic Resilience Walkthrough (Current) - call_chain_id is always generated per execution: uuid.uuid4().hex. - request_id is optional; the effective request id used for jitter is: working_request_id = request_id if request_id else call_chain_id Deterministic backoff jitter: - attempt_index is 0-based and used consistently for both backoff exponent and jitter seed. - base_backoff = min(max_delay_s, base_delay_s * (multiplier ** attempt_index)) - seed = sha256(f"{provider_id}:{model_id}:{working_request_id}:{attempt_index}") - jitter_factor = 0.5 + 0.5 * rng.random() # [0.5, 1.0) - sleep_time = min(max_delay_s, base_backoff * jitter_factor) Telemetry: - total_attempts counts actual provider invocations only (network calls). - call_chain_completed is emitted exactly once via finally, with: call_chain_id, request_id(working_request_id), duration_ms,
```

```
total_attempts, final_provider,
terminal_state(success|exhausted|fatal_error) Verification: -
Deterministic jitter tests confirm identical sleep sequences for same
request_id and different sequences for different request_id. - Telemetry
tests confirm call_chain_completed emission and correct attempt counting.
- Circuit breaker half-open regression test still passes.
```

If you want me to critique the *other review* line-by-line (what it said vs what's true), paste it cleanly without the "Good/Bad" header and I'll mark each sentence as ✅ accurate / ❌ drift / ⚠️ ambiguous and replace the drifted ones with corrected language.